

Lecture 06: Object Oriented Design - Heuristics and Coffee

Course Topic: Object-Oriented Design (OOD) Case Study

Core Material: The Mark IV Special Coffee Maker Design Problem

Line Spacing: 1.15

1. Introduction to the OOD Case Study

Over past teaching experiences with professional software developers, a common pattern of design mistakes emerges when students transition from basic conceptual understanding to real-world software architecture. This lecture focuses on exploring these common architectural pitfalls and walks through a complete, clean object-oriented solution for an embedded real-time system controlling a coffee maker.

On the first phase of OOD training, foundational concepts are established: classes, objects, relationships, methods, polymorphism, and basic Unified Modeling Language (UML) notation. The Mark IV Special Coffee Maker problem is utilized as a core exercise to test these concepts in practice.

2. Hardware Specification: The Mark IV Special Coffee Maker

The Mark IV Special makes up to 12 cups of coffee at a time. The physical workflow and requirements are specified as follows:

- **Brewing Workflow:** The user places a filter in the filter holder, fills it with coffee grounds, and slides the holder into its receptacle. Up to 12 cups of water are poured into the strainer. When the user presses the "Brew" button, the water heats until boiling. Steam pressure forces the water to spray over the grounds, dripping through the filter into the pot.
- **Warmer Plate Policy:** The pot is kept warm by a warmer plate. Crucially, the warmer plate must *only* turn on if there is coffee in the pot.
- **Interruption/Safety Policy:** If the pot is removed from the warmer plate while water is actively being sprayed over the grounds, the water flow must be stopped immediately to prevent brewed coffee from spilling onto the warmer plate.

Hardware Components to Monitor and Control

- **Boiler Heating Element:** Can be turned ON or OFF.
- **Warmer Plate Heating Element:** Can be turned ON or OFF.
- **Warmer Plate Sensor:** Detects three distinct states: warmerEmpty, potEmpty, or potNotEmpty.
- **Boiler Sensor:** A float switch determining water presence with two states: boilerEmpty or boilerNotEmpty.

- **Brew Button:** A momentary button that starts the brewing cycle. Contains an indicator light that illuminates when the brewing cycle completes and the coffee is ready.
- **Pressure-Relief Valve:** Opens to reduce boiler pressure. Dropping the pressure stops the water flow to the filter. Can be commanded to be OPEN or CLOSED.

The Low-Level Hardware API

The hardware engineering team provided a low-level Java interface (CoffeeMakerAPI.java) to interact with the device drivers without requiring bit-twiddling I/O code:

```
public interface CoffeeMakerAPI {
    public static CoffeeMakerAPI api = null; // Set by main

    public int getWarmerPlateStatus();
    public static final int WARMER_EMPTY = 0;
    public static final int POT_EMPTY = 1;
    public static final int POT_NOT_EMPTY = 2;

    public int getBoilerStatus();
    public static final int BOILER_EMPTY = 0;
    public static final int BOILER_NOT_EMPTY = 1;

    public int getBrewButtonStatus();
    public static final int BREW_BUTTON_PUSHED = 0;
    public static final int BREW_BUTTON_NOT_PUSHED = 1;

    public void setBoilerState(int boilerStatus);
    public static final int BOILER_ON = 0;
    public static final int BOILER_OFF = 1;

    public void setWarmerState(int warmerState);
    public static final int WARMER_ON = 0;
    public static final int WARMER_OFF = 1;

    public void setIndicatorState(int indicatorState);
    public static final int INDICATOR_ON = 0;
    public static final int INDICATOR_OFF = 1;

    public void setReliefValveState(int reliefValveState);
    public static final int VALVE_OPEN = 0;
    public static final int VALVE_CLOSED = 1;
}
```

3. Analyzing Common Architectural Errors (The Hideous Solution)

A typical initial architectural model submitted by beginners features a central CoffeeMaker class surrounded by precise physical mirrors of the hardware parts: Boiler, WarmerPlate, Button, and Light, with Sensor and Heater base classes. While intuitive on the surface, this model displays fatal design flaws:

Design Error Type	Description & Analysis
Missing Methods	The diagrams completely lack behavioral methods. Partitioning software based purely on physical components rather than behavior is a critical error. System behavior must drive the architectural split.
Vapor Classes	Classes like Light or Button contain no internal variables or independent state. They act merely as call translators delegating directly to the low-level API. They provide no useful abstractions and can be completely eliminated without changing logic.
Imaginary Abstractions	Interfaces like Heater and Sensor have no active users or clients in the system. Furthermore, the contract of Sensor.sense() is ambiguous because the boiler sensor returns 2 states while the warmer plate returns 3 states, making the interface contract weak and useless.
God Classes	Because the peripheral classes are "vapor", all true system intelligence and coordination logic collapse back into a single monolithic class (CoffeeMaker). This defeats the primary OOD goal of distributing behavior into distinct, manageable parts.

4. The Object-Oriented Solution: The Abstract Model

To fix the architecture, developers must separate physical implementation details from the essential nature of the domain. The fundamental abstract problem is: *How do you make coffee?* It requires a hot water source delivering water over grounds into a collection container.

The Core Domain Abstractions

- **HotWaterSource:** Responsible for heating water and delivering it over the filter grounds.
- **ContainmentVessel:** Responsible for retaining the brewed coffee, keeping it warm, and reporting content availability.
- **UserInterface:** Responsible for sensing user intent (button presses) and reporting status

outputs (indicator light).

Crucial Rule (Crossed Wires Avoidance): Associations in UML represent software message-passing pathways, not physical material flows. Just because physical coffee flows from the boiler to the pot does not mean HotWaterSource points to ContainmentVessel. In software behavior, the ContainmentVessel actually sends status updates to the HotWaterSource, meaning the structural dependency runs backward compared to the physical domain.

5. Applying the Dependency Inversion Principle (DIP)

To prevent high-level coffee-making policies from depending on low-level Mark IV hardware specifications, we apply the **Dependency Inversion Principle (DIP)**. The core abstractions (UserInterface, HotWaterSource, ContainmentVessel) remain completely pure, abstract, and unaware of the Mark IV API. All hardware-specific behaviors are deferred to subclasses.

Abstract Policies vs. Concrete Derivatives

- **High-Level Policy:** Checking if both the water source and containment vessel are ready before allowing a brew cycle to initiate is a high-level policy handled entirely inside the abstract UserInterface parent class.
- **Low-Level Detail:** Calling `api.getBoilerStatus()` or `api.getWarmerPlateStatus()` is an implementation detail encapsulated inside concrete derivatives (M4HotWaterSource and M4ContainmentVessel).

The Multithreading vs. Polling Dilemma

Decisions regarding real-time synchronization (whether to use an operating system thread per sensor or loop-based polling) are completely irrelevant to high-level OOD. A clean design assumes asynchronous message passing. For simple architectural environments lacking threading capabilities, a clean Pollable interface can be implemented by concrete derivatives, allowing a centralized execution loop to drive the system cleanly.

6. Complete Clean Code Architecture Implementation

Below is the complete, cohesive Java implementation separating high-level policies from Mark IV hardware details.

Core Abstractions (High-Level Policy)

```
// Pollable.java
public interface Pollable {
    public void poll();
}

// UserInterface.java
public abstract class UserInterface {
    private HotWaterSource hws;
```

```

private ContainmentVessel cv;
protected boolean isComplete;

public UserInterface() {
    isComplete = true;
}

public void init(HotWaterSource hws, ContainmentVessel cv) {
    this.hws = hws;
    this.cv = cv;
}

public void complete() {
    isComplete = true;
    completeCycle();
}

protected void startBrewing() {
    if (hws.isReady() && cv.isReady()) {
        isComplete = false;
        hws.start();
        cv.start();
    }
}

public abstract void done();
public abstract void completeCycle();
}

// HotWaterSource.java
public abstract class HotWaterSource {
    private UserInterface ui;
    private ContainmentVessel cv;
    protected boolean isBrewing;

    public HotWaterSource() {
        isBrewing = false;
    }

    public void init(UserInterface ui, ContainmentVessel cv) {
        this.ui = ui;
        this.cv = cv;
    }

    public void start() {
        isBrewing = true;
        startBrewing();
    }
}

```

```

    public void done() {
        isBrewing = false;
    }

    protected void declareDone() {
        ui.done();
        cv.done();
        isBrewing = false;
    }

    public abstract boolean isReady();
    public abstract void startBrewing();
    public abstract void pause();
    public abstract void resume();
}

// ContainmentVessel.java
public abstract class ContainmentVessel {
    private UserInterface ui;
    private HotWaterSource hws;
    protected boolean isBrewing;
    protected boolean isComplete;

    public ContainmentVessel() {
        isBrewing = false;
        isComplete = true;
    }

    public void init(UserInterface ui, HotWaterSource hws) {
        this.ui = ui;
        this.hws = hws;
    }

    public void start() {
        isBrewing = true;
        isComplete = false;
    }

    public void done() {
        isBrewing = false;
    }

    protected void declareComplete() {
        isComplete = true;
        ui.complete();
    }
}

```

```

        protected void containerAvailable() {
            hws.resume();
        }

        protected void containerUnavailable() {
            hws.pause();
        }

        public abstract boolean isReady();
    }

```

Mark IV Concrete Subclasses (Low-Level Infrastructure)

```

// M4UserInterface.java
public class M4UserInterface extends UserInterface implements Pollable
{
    private CoffeeMakerAPI api;

    public M4UserInterface(CoffeeMakerAPI api) {
        this.api = api;
    }

    public void poll() {
        int buttonStatus = api.getBrewButtonStatus();
        if (buttonStatus == api.BREW_BUTTON_PUSHED) {
            startBrewing();
        }
    }

    public void done() {
        api.setIndicatorState(api.INDICATOR_ON);
    }

    public void completeCycle() {
        api.setIndicatorState(api.INDICATOR_OFF);
    }
}

// M4HotWaterSource.java
public class M4HotWaterSource extends HotWaterSource implements
Pollable {
    private CoffeeMakerAPI api;

    public M4HotWaterSource(CoffeeMakerAPI api) {
        this.api = api;
    }
}

```

```

    public boolean isReady() {
        int boilerStatus = api.getBoilerStatus();
        return boilerStatus == api.BOILER_NOT_EMPTY;
    }

    public void startBrewing() {
        api.setReliefValveState(api.VALVE_CLOSED);
        api.setBoilerState(api.BOILER_ON);
    }

    public void poll() {
        int boilerStatus = api.getBoilerStatus();
        if (isBrewing) {
            if (boilerStatus == api.BOILER_EMPTY) {
                api.setBoilerState(api.BOILER_OFF);
                api.setReliefValveState(api.VALVE_CLOSED);
                declareDone();
            }
        }
    }

    public void pause() {
        api.setBoilerState(api.BOILER_OFF);
        api.setReliefValveState(api.VALVE_OPEN);
    }

    public void resume() {
        api.setBoilerState(api.BOILER_ON);
        api.setReliefValveState(api.VALVE_CLOSED);
    }
}

// M4ContainmentVessel.java
public class M4ContainmentVessel extends ContainmentVessel implements
Pollable {
    private CoffeeMakerAPI api;
    private int lastPotStatus;

    public M4ContainmentVessel(CoffeeMakerAPI api) {
        this.api = api;
        lastPotStatus = api.POT_EMPTY;
    }

    public boolean isReady() {
        int plateStatus = api.getWarmerPlateStatus();
        return plateStatus == api.POT_EMPTY;
    }
}

```



```

public void poll() {
    int potStatus = api.getWarmerPlateStatus();
    if (potStatus != lastPotStatus) {
        if (isBrewing) {
            handleBrewingEvent(potStatus);
        } else if (isComplete == false) {
            handleIncompleteEvent(potStatus);
        }
        lastPotStatus = potStatus;
    }
}

private void handleBrewingEvent(int potStatus) {
    if (potStatus == api.POT_NOT_EMPTY) {
        containerAvailable();
        api.setWarmerState(api.WARMER_ON);
    } else if (potStatus == api.WARMER_EMPTY) {
        containerUnavailable();
        api.setWarmerState(api.WARMER_OFF);
    } else { // POT_EMPTY
        containerAvailable();
        api.setWarmerState(api.WARMER_OFF);
    }
}

private void handleIncompleteEvent(int potStatus) {
    if (potStatus == api.POT_NOT_EMPTY) {
        api.setWarmerState(api.WARMER_ON);
    } else if (potStatus == api.WARMER_EMPTY) {
        api.setWarmerState(api.WARMER_OFF);
    } else { // POT_EMPTY
        api.setWarmerState(api.WARMER_OFF);
        declareComplete();
    }
}
}

```

System Entry Point

```

// CoffeeMaker.java
public class CoffeeMaker {
    public static void main(String[] args) {
        CoffeeMakerAPI api = new M4CoffeeMakerAPIImplementation(); //
Concrete driver implementation
        M4UserInterface ui = new M4UserInterface(api);
        M4HotWaterSource hws = new M4HotWaterSource(api);
        M4ContainmentVessel cv = new M4ContainmentVessel(api);
    }
}

```

```

// Cross-wiring references clean initialization
ui.init(hws, cv);
hws.init(ui, cv);
cv.init(ui, hws);

// Infinite polling execution driving the state machine loop
while (true) {
    ui.poll();
    hws.poll();
    cv.poll();
}
}
}

```

7. Architectural Evaluation & Pragmatic Trade-offs

The structured abstract model isolates dependencies perfectly. All dependencies point inward across the abstraction boundaries. The core logic can be seamlessly reused to build an automated tea brewer, a vending machine, or a chicken soup maker without modifications to the high-level policy code.

However, an honest pragmatic evaluation reveals an **"OOverkill"** phenomenon for small systems. If implemented as a procedural Finite State Machine (FSM), the program would require only 7 states, 18 transition definitions, and could be fully written in under a single page of code. The OO approach takes five pages of code. In tiny real-time embedded environments, the architectural cost of OOD abstraction may outweigh its decoupling benefits. However, as system scale, complexity, and variance requirements grow, the management of dependencies via clean OOD interfaces becomes completely essential to control technical debt.